Reproduction of Kim's Convolutional Neural Network for Sentence Classification on SST-2

Chiranjivi Panda
Course: Deep Learning
24cs2013@rgipt.ac.in

***Abstract*—We re-implement the sentence classification model of Kim (2014) in PyTorch and evaluate it on the binary SST-2 sentiment task. The model turns a sentence into a matrix of pre-trained word vectors, applies convolutional filters of widths 3, 4, and 5 to detect short word patterns, keeps the strongest activation of each filter, and classifies the pooled features with a linear layer. We compare four embedding settings and run one ablation on the filter configuration. Our best accuracy is 85.0% with fine-tuned embeddings, compared with 87.2% in the paper. We describe the differences between our setup and the paper so that the gap can be interpreted honestly.**

I. Problem and Motivation

Sentence classification is the task of reading a short piece of text and assigning it a label. Our label set is binary: positive or negative sentiment. The dataset is SST-2, built from movie reviews.

Before Kim's paper, many strong text models used hand-designed features or complex structures such as parse trees. Kim showed that a very simple convolutional neural network, applied on top of pre-trained word vectors, works well. His main message is that *pre-trained word vectors are strong general-purpose features*, and that a small CNN can turn them into a competitive classifier with little tuning.

Our goal is to re-implement this model in PyTorch, reproduce the SST-2 part of the paper's results at a small scale, and check one design choice with an ablation.

II. Method in Plain Language

A sentence is first split into words. Each word is replaced by a vector of 300 numbers. These vectors come from Google's word2vec, which was trained on a large amount of news text and captures word meaning as numbers. A sentence of n words becomes a matrix of shape $n \times 300$.

A convolutional filter of width h looks at h neighbouring words at a time. The filter has its own weights $\mathbf{w}$ and bias b. For a window starting at position i, we concatenate the word vectors in the window and compute a single number:

$$c_i = \mathrm{ReLU}\big(\mathbf{w}^\top \mathbf{x}_{i:i+h-1} + b\big)\,. \tag{1}$$

Intuitively, this number is high when the window matches the pattern the filter has learned to look for.

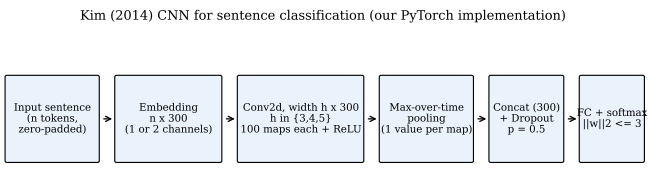


Fig. 1. Our implementation of Kim's sentence CNN. The sentence is turned into a matrix of word vectors, convolutional filters of widths 3, 4, and 5 look at short word windows, max-pooling keeps the strongest response of each filter, and a linear layer produces the two class scores.

Sliding the filter over the whole sentence gives a list of numbers $c_1, c_2, \ldots$ Equation (1) is applied at every window. We then keep only the maximum value:

$$\hat{c} = \max_i c_i. \tag{2}$$

This step is called *max-over-time pooling*. It answers the question "did this pattern appear anywhere in the sentence, and how strongly?".

We use several filters of each width 3, 4, and 5, with 100 filters per width. This gives 300 pooled features per sentence. These 300 numbers are concatenated, passed through dropout for regularization, and fed into a linear layer that outputs two numbers: a score for "negative" and a score for "positive".

We compare four variants:

- **rand**: word vectors are initialised randomly and learned during training.
- **static**: word vectors come from word2vec and are frozen.
- **non-static**: word vectors come from word2vec and are fine-tuned during training.
- **multichannel**: two copies of word2vec vectors; one copy is frozen, the other is fine-tuned.

Figure 1 shows the whole model.

III. Implementation Details

Dataset. We use SST-2 (binary sentiment). We follow the paper and label reviews with ratings 0–1 as negative and 3–4 as positive, dropping neutral examples. The paper trains on both full sentences and their sub-phrases from the Stanford Sentiment Treebank. We do the same, with duplicate phrase strings removed. All text is lower-cased and tokenised. Table I

TABLE I
SST-2 DATASET STATISTICS AFTER OUR PREPROCESSING VERSUS THE PAPER (KIM 2014, TABLE 1).

Statistic	Ours	Paper
Classes	2	2
Avg. sentence length (dev+test)	18.6	19
Avg. length of training phrases	8.8	n/a
Train examples	71,507	6,920+
Dev sentences	872	872
Test sentences	1,821	1,821
$\lvert V \rvert$ (all splits)	17,240	16,185
$\lvert V_{\text{pre}} \rvert$ (in word2vec)	15,756	14,838
Pre-trained coverage	91.4%	91.7%

TABLE II
SST-2 TEST ACCURACY. OUR VALUES ARE MEAN ± STANDARD DEVIATION OVER THREE SEEDS; KIM REPORTS ONE VALUE PER MODEL.

Model	Ours (%)	Kim (%)	Δ (pp)
CNN-rand	75.1 ± 0.7	82.7	-7.6
CNN-static	83.7 ± 0.6	86.8	-3.1
CNN-non-static	85.0 ± 1.0	87.2	-2.2
CNN-multichannel	84.6 ± 0.4	88.1	-3.5

compares our dataset statistics with the paper's Table 1 and shows that our preprocessing produces very similar numbers.

Vocabulary and embeddings. We build the vocabulary from the training data only. Words not in word2vec are initialised with a small uniform distribution whose variance matches that of the pre-trained vectors.

Model. The core module is a PyTorch class that defines the embedding layer(s), three `Conv2d` layers (one per filter width), a dropout layer, and a linear output layer. The convolution kernel has shape $(h, 300)$, so each filter spans the full embedding dimension and slides along the sentence.

Training. We use batch size 50, the Adadelta optimizer ($\rho = 0.95$), dropout rate 0.5, and the paper's constraint that each row of the final linear layer has L2 norm at most 3. We train for at most 15 epochs and stop early if the development accuracy does not improve for four epochs. The reported test accuracy is the test accuracy at the epoch of best development accuracy.

Compute and seeds. All runs use a single NVIDIA T4 GPU. Main results are averaged over three seeds (42, 43, 44). The ablation uses two seeds (42, 43) for compute reasons.

IV. RESULTS VERSUS THE PAPER

Table II shows our SST-2 test accuracy next to Kim's reported values, and Fig. 2 visualises the same comparison. Across three seeds, the pattern matches the paper: using pre-trained word vectors helps a lot, and fine-tuning them helps a little more. The multichannel variant does not improve over the single-channel fine-tuned model in our runs; the paper itself calls this comparison "mixed".

Our absolute numbers are a few points below the paper. We discuss likely reasons in Section VI. Table III shows the per-seed values behind the mean accuracies.

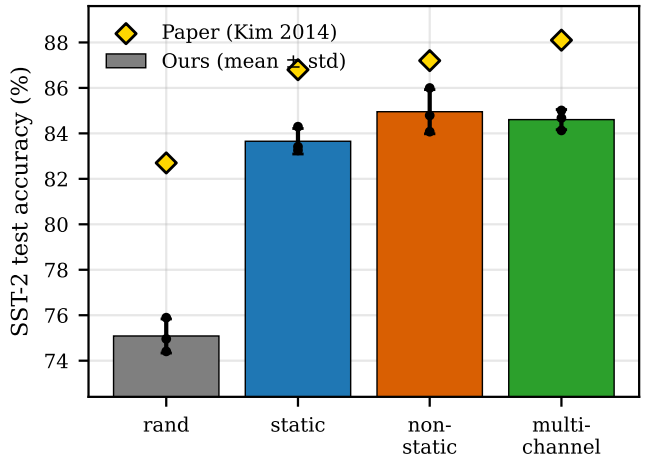


Fig. 2. SST-2 test accuracy for the four embedding settings. Bars show our mean over three seeds with standard-deviation error bars; dots show individual seed results; diamonds mark Kim's reported values. The ordering of the four variants matches the paper, but all absolute numbers are a few points lower.

TABLE III
PER-SEED DEVELOPMENT AND TEST ACCURACY (%) AND THE SELECTED BEST-DEVELOPMENT EPOCH FOR EACH VARIANT.

Model	Seed	Dev	Test	Epoch
CNN-rand	42	74.08	74.41	4
CNN-rand	43	73.17	74.96	8
CNN-rand	44	78.44	75.89	13
CNN-static	42	83.60	83.25	2
CNN-static	43	83.60	84.29	3
CNN-static	44	82.57	83.42	2
CNN-non-static	42	84.40	84.79	10
CNN-non-static	43	83.26	84.07	3
CNN-non-static	44	85.44	86.00	12
CNN-multichannel	42	84.29	85.01	4
CNN-multichannel	43	83.37	84.13	3
CNN-multichannel	44	83.83	84.68	3

V. ABLATION: FILTER CONFIGURATION

We study how the filter widths and the number of filters affect accuracy, using the fine-tuned (non-static) model. Three settings are compared in Table IV and Fig. 3, each averaged over two seeds.

The full multi-width model is better than the small width-3 model. However, a width-3 model with 300 filters scores slightly higher than the multi-width model with 300 filters in total. We therefore cannot claim that *having multiple widths* is itself the reason for the improvement over the small model; adding filters of a single width also helps. With only two seeds, these differences are small and should be read as descriptive, not conclusive.

VI. FAILURE CASES AND WHAT DID NOT WORK

Our best model is about 2 percentage points below the paper on SST-2. We can list likely reasons but cannot isolate a single cause from this experiment.

Random baseline is not a clean comparison. In our `rand` setting, the embedding layer uses PyTorch's default

TABLE IV
FILTER-CONFIGURATION ABLATION ON SST-2 WITH FINE-TUNED EMBEDDINGS. MEAN ± STANDARD DEVIATION OVER TWO SEEDS.

Filter configuration	Accuracy (%)
Width 3, 100 filters	83.69 ± 0.39
Widths 3, 4, 5, 100 filters each	84.43 ± 0.50
Width 3, 300 filters	84.82 ± 0.35

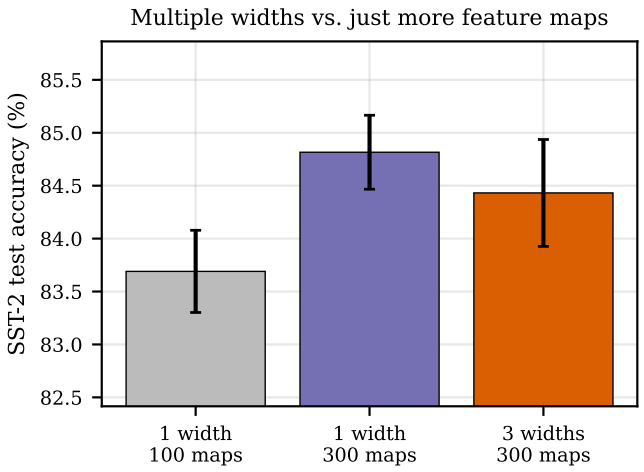


Fig. 3. Multi-width model versus a single-width model with more filters. Adding filters at a single width recovers most of the gain, so our experiment does not isolate "multiple widths" as the cause of the improvement.

initialisation, which produces values on a much larger scale than the variance-matched initialisation used for unknown words in the other variants. The gap between `rand` and `static` in our results is therefore partly a scale effect, not purely the effect of pre-training. We still report the number but do not use it to measure "how much pre-training helps".

Preprocessing differences. Our tokenisation, lower-casing, and phrase de-duplication rules are not identical to the paper, which does not fully specify them. Any such difference can move the final number by a point or two.

Vocabulary built from training data only. Words seen only in development or test sentences map to a single `<UNK>` vector. A small vocabulary-coverage check shows this affects about 4–5% of tokens in dev and test.

Where the model fails. Figure 4 breaks the test set into simple subsets. Accuracy is lower on long sentences (78.0% for sentences longer than 30 tokens) and on sentences containing the word "but" (82.3% versus 85.2% without "but"). This is consistent with the known limitation of max-pooling CNNs: they detect local patterns well but do not explicitly reason about long-range structure or contrast.

What did not help. The multichannel variant did not beat the fine-tuned single-channel model in our runs. We also observed that the L2-norm constraint on the output weights was never active in our training (the norms stayed below the threshold of 3), so this regulariser had no measurable effect in our setup.

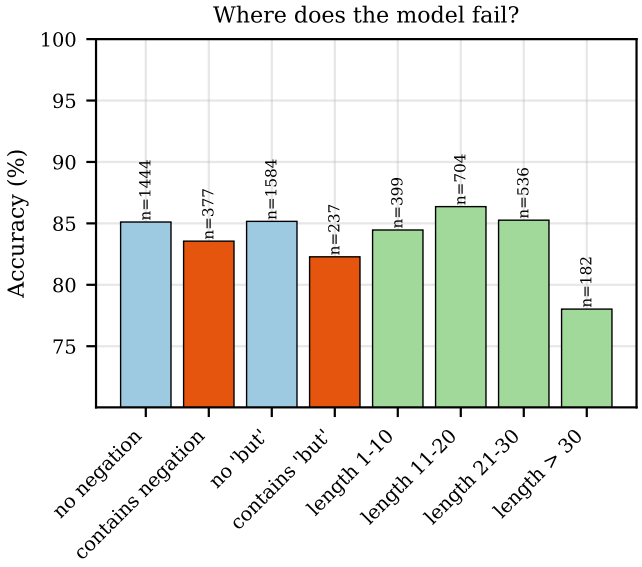


Fig. 4. Test accuracy of the fine-tuned model on simple subsets of the SST-2 test set. The weakest subsets are long sentences and sentences containing "but". Numbers above each bar show the subset size.

VII. HONEST SUMMARY

We reproduced the SST-2 part of Kim (2014) at small scale in PyTorch. The qualitative story of the paper holds in our experiments: pre-trained vectors give a large improvement over random initialisation, and fine-tuning gives a small additional improvement. Our absolute numbers are a few points below the paper, mostly because of unspecified preprocessing details and initialisation choices in our `rand` baseline. We ran one focused ablation on the filter configuration, and we describe its limits rather than overclaiming.

REFERENCES

[1] Y. Kim, "Convolutional Neural Networks for Sentence Classification," in *Proc. EMNLP*, 2014.

[2] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed Representations of Words and Phrases and their Compositionality," in *Proc. NeurIPS*, 2013.

[3] R. Socher *et al.*, "Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank," in *Proc. EMNLP*, 2013.

[4] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Proc. NeurIPS*, 2019.